DJANGO 6.1 • FIELD OPERATIONS RUNBOOK

Project Manager: Inventory, BOM & Assembly Guide

Category Prefixes, Auto Part Numbering, Datasheets/Scans, Bill of Materials (BOM) & ODS Importer

HOST: **localhost:8000**
DB: **MySQL (Project_manager)**
THEME: **Cadmium Green (#137547)**
RELEASE: **v2.1 Production**

1 COMPREHENSIVE CONTROLLER & ROUTE REGISTRY

38 ACTIVE ENDPOINTS

DOMAIN / MODEL	URL PATTERN	CLASS-BASED VIEW	TEMPLATE RESOURCE	ACTION
Dashboard	`/`	`views.home`	`templates/home.html`	KPI OVERVIEW
Part Catalog	`/parts/` | `/parts/add/`	`PartListView` | `PartCreateView`	`core/part_list.html` | `form.html`	SEARCH/CREATE
	`/parts/<pk>/`	`PartDetailView`	`core/part_detail.html`	STOCK SPECS
	`/parts/<pk>/edit/` | `/delete/`	`PartUpdateView` | `DeleteView`	`part_form.html` | `confirm_delete.html`	EDIT / PURGE
Bills of Materials (BOM)	`/boms/` | `/boms/add/`	`BOMListView` | `BOMCreateView`	`core/bom_list.html` | `bom_form.html`	ASSEMBLIES
	`/boms/<pk>/`	`BOMDetailView`	`core/bom_detail.html`	COST ROLLUP
	`/boms/<pk>/edit/` | `/delete/`	`BOMUpdateView` | `BOMDeleteView`	`bom_form.html` | `confirm_delete.html`	EDIT / DELETE
BOM Line Items	`/boms/<bom_pk>/items/add/` | `/bom-items/<pk>/delete/`	`BOMItemCreateView` | `BOMItemDeleteView`	`core/bom_item_form.html` | `confirm_delete.html`	COMPONENT LINK
Documents & Scans	`/parts/<part_pk>/documents/add/`	`PartDocumentCreateView`	`core/part_document_form.html`	ATTACH FILE
	`/documents/<pk>/` | `/edit/` | `/delete/`	`DetailView` | `UpdateView` | `DeleteView`	`part_document_detail.html`	HIGH-RES VIEW
Category	`/categories/` | `/add/` | `/<pk>/`	`CategoryListView` | `CreateView` | `DetailView`	`category_list.html` | `detail.html`	TAXONOMY
Company / Suppliers	`/companies/` | `/add/` | `/<pk>/`	`CompanyListView` | `CreateView` | `DetailView`	`company_list.html` | `detail.html`	SUPPLIERS
Manufacturer	`/manufacturers/` | `/add/` | `/<pk>/`	`ManufacturerListView` | `CreateView`	`manufacturer_list.html`	OEM INDEX
Storage Location	`/locations/` | `/add/` | `/<pk>/`	`LocationListView` | `CreateView`	`location_list.html`	STORAGE BINS

2 CATEGORY PREFIX & TAXONOMY

CORE/MODELS.PY

- **Prefix Field**: `Category.prefix` (e.g. `100` for Passives, `200` for Compute/RF, `300` for Connectors).
- **Recursive Inheritance**: `get_prefix()` checks current category; if empty, ascends parent tree (`parent.get_prefix()`).
- **Dropdown Display**: `Category.__str__()` formats hierarchies: `[100] Passives > Resistors > SMD 0805`.
- **Taxonomy Nesting**: Self-referential `parent = ForeignKey('self')` allows unlimited subcategory depth.

> **Subcategory Inheritance:** Defining `100` on *Passives* allows *Resistors* and *Capacitors* to inherit prefix `100` without re-typing.

3 AUTOMATED PART NUMBERING

PREFIX ENGINE

- **Starting Sequence**: Sequential numbers start at `1001` (e.g. `100-1001`, `100-1002`, ..., `100-1234`).
- **Dynamic Increment**: `generate_next_part_number(category)` queries existing parts with `prefix-*`, parses numeric suffixes, and takes `max_num + 1`.
- **Intelligent Formatting in `save()`**:
  - Blank → Auto-generates next available sequence (`100-1001`).
  - Digits only (e.g. `1234`) → Prepends prefix: `100-1234`.
  - Custom string → Preserved untouched.
- **Collision Avoidance**: Loops candidate numbers until verified unused.

4 BILL OF MATERIALS (BOM) & PRODUCT ASSEMBLY ARCHITECTURE

BOM & BOMITEM MODELS

- **Assembly Tracking (BOM)**: Represents fully assembled products and sub-assemblies (e.g. String, Node, Base Station, Tipups).
- **Assembly Instructions**: Integrated `CKEditor5Field` for WYSIWYG build instructions, test protocols, and revision notes.
- **Real-Time Rollup Metrics**:
  - `total_items_count`: Unique component lines in the build.
  - `total_parts_quantity`: Total raw physical pieces required.
  - `total_cost`: Sum of `(item.part.price_per * item.quantity)`.
- **Component Links (BOMItem)**: Foreign key linking `BOM` to `Part` with:
  - `quantity`: Pieces required per single finished assembly.
  - `reference_designator`: PCB / schematic designators (e.g. `U1`, `C1-C4`, `BAR1`).
  - `notes`: Solder, trimming, or torque instructions.
- **Subtotal Calculation**: `subtotal` dynamically returns line cost based on live catalog pricing.

ASSEMBLY NAME	UNIQUE COMPONENTS	TOTAL PARTS COUNT	TOTAL UNIT COST	SAMPLE REFERENCE DESIGNATORS
String Assembly	9 line items	72 pieces	**$153.17**	`5 nodes baseline, wiring harness, terminal blocks`
Sensor Node Assembly	8 line items	8 pieces	**$14.34**	`Enclosure, sensory probe, interconnect, seal`
Base Station Assembly	5 line items	5 pieces	**$101.04**	`Compute core, RF transceiver, antenna, power supply`
Tipups Wireless Monitor	12 line items	12 pieces	RFQ PENDING	`U1 (Micro), U2 (RF), BAR1, BT1, C1-C6, ANT1`

5 ODS SPREADSHEET INGESTION ENGINE (BOMS.ODS)

MANAGE.PY IMPORT_BOMS

- **Zero-Dependency Parser**: Built directly on Python standard library (`zipfile` + `xml.etree.ElementTree`) for pure ODS XML parsing.
- **Master Parts List Extraction**: Reads `Parts list` sheet (24 parts), maps suppliers, unit price, quantity, confirmed flag, and external order URLs.
- **DigiKey Component Ingestion**: Parses `tipups` sheet (12 components) creating parts with manufacturers and PCB designators.
- **Automated Revision Superseding**:
  - Detects revision series (e.g. `2002.1` → `2002.2`).
  - Populates `superseded_by` Foreign Key relationship automatically.
- **Automated Assembly Construction**: Generates assemblies (`string`, `Node`, `Base_station`, `tipups`) and resolves component references.

```
# Execute ODS Import Command (Idempotent & Safe to Re-run) python manage.py import_boms Reading ODS file:
/home/michael/Desktop/Project_manager/BoMs.ods... Found 7 sheets: string, Node, Base_station, Signal_strength, String tester, tipups, Parts
list ✓ Linked: 2002.1 is superseded by -> 2002.2 ✓ Linked: 2003.1 is superseded by -> 2003.2 Total Parts in DB: 36 | Total Companies in DB: 11
| Total BOMs in DB: 4 | Total BOM Items: 34
```

6 DATASHEETS, SCANS & ARTICLES

PARTDOCUMENT

- **Relationship**: One-to-Many with `Part` (`part.documents.all()`). Multiple attachments per part number.
- **Document Types**: `datasheet` (PDF) • `pinout` (Diagram) • `scan` (Drawing) • `instructions` (Assembly) • `article` (Notes)
- **File Storage**: Uploads to `media/parts/documents/%Y/%m/`. Auto-detects `is_image`, `is_pdf`, and formats file size.
- **WYSIWYG Articles**: `content = CKEditor5Field()` for rich pinout tables, troubleshooting procedures, and notes.

7 THEME & UI ARCHITECTURE

CADMIUM GREEN

- **Base Palette**: Integrated `bootstrapcolors-custom.css`: Cadmium Green (`#137547`), Forest (`#054a29`), Mint accents.
- **Responsive Sidebar**: Permanent desktop drawer; collapses into touch-friendly offcanvas on mobile devices with BOM nav.
- **Dual Dark/Light Theme**: Bootstrap 5.3 `data-bs-theme` with persistent client-side `localStorage` sync.
- **Interactive Calculators**: Real-time JS total cost calculator & dynamic category prefix placeholders.

8 OPERATOR WALKTHROUGH & TERMINAL RUNBOOK

STANDARD PROCEDURE

```
# 1. Boot local development server source .venv/bin/activate python manage.py runserver 127.0.0.1:8000 # 2. Ingest or Re-synchronize BoMs.ods Spreadsheet python manage.py import_boms # 3. Create Category with Prefix (e.g. 100 for Passives) curl -X POST http://127.0.0.1:8000/categories/add/ -d "name=Passives&prefix=100&description=Passive Components" # 4. Create Part leaving part_number blank -> auto-generates 100-1001 curl -X POST http://127.0.0.1:8000/parts/add/ -d "description=10uF Capacitor&category=1&quantity=100&price_per=0.15" # -> Generated: part_number = '100-1001' | price = 15.00 # 5. Attach a Datasheet / Scan to the component curl -X POST http://127.0.0.1:8000/parts/1/documents/add/ \ -F "title=TI LM317 Datasheet" \ -F "document_type=datasheet" \ -F "file=@datasheet.pdf" # 6. Add a Line Item to a Bill of Materials Assembly curl -X POST http://127.0.0.1:8000/boms/1/items/add/ \ -d "part=1&quantity=5&reference_designator=C1-C5¬es=Decoupling capacitors"
```

OPERATIONAL PHASE	ACTION PERFORMED IN INTERFACE	SYSTEM REACTION & STATE CHANGE
1. ODS Ingestion	Place `BoMs.ods` in root and run `python manage.py import_boms`.	Ingests 36 parts, 11 companies, 4 BOM assemblies, and links superseded revisions.
2. Category Setup	Go to `/categories/add/`. Enter Name & Prefix (e.g. `100`).	Prefix stored; inherited by any child subcategories via `get_prefix()`.
3. Part Registration	Select category on `/parts/add/`. Leave part number blank.	Auto-generates `100-1001`. Computes `price = price_per * qty`.
4. Suffix Override	Enter `1234` in Part Number input.	Auto-prepends category prefix to produce `100-1234`.
5. BOM Management	Go to `/boms/` → Click *Add BOM* or inspect existing assembly.	Displays assembly summary, total unit cost, and component list.
6. Add Component to BOM	On BOM detail page, click *Add Component*. Select part & quantity.	Line item added; rolls up `total_cost` and `total_parts_quantity`.
7. Attach Resources	On Part detail page, click *Add Document / Scan*.	Uploads PDF/scan, renders thumbnail, and attaches to part record.
8. Document Viewing	Click *View* on any document card.	Opens dedicated high-res viewer, embedded PDF, or article body.